

Codeforces Round 1122 (Div. 3)

A. Good Contest

1 second, 256 megabytes

The next programming contest has three problems and n participants.

Problem 1 is easy, problem 2 is medium, and problem 3 is hard.

A participant is called *weak* if they did not solve all three problems.

Unfortunately, the scoreboard was lost. The only remaining information is an array a of length 3, where a_i is the number of participants who solved problem i .

Among all scoreboards consistent with this information, find the minimum possible number of weak participants.

Input

The first line contains an integer t ($1 \leq t \leq 3000$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 9$) — the number of participants.

The second line of each test case contains three integers a_1, a_2, a_3 ($0 \leq a_i \leq n$), where a_i is the number of participants who solved problem i .

Output

For each test case, print a single integer — the minimum possible number of weak participants.

input
6 3 3 3 3 4 4 4 3 1 1 1 1 9 9 8 9 5 0 5 5 6 4 3 2
output
0 1 0 1 5 4

In the first test case, all 3 participants can have solved all three problems, so the answer is 0.

In the second test case, participant 1 could have solved only problems 1 and 2, while participants 2, 3, and 4 solved all three problems. Therefore, participant 1 is the only participant who is weak, so the answer is 1. It can be shown that this is minimal.

B. Three Piles

1 second, 256 megabytes

Alice and Bob are playing a game with three piles of stones. Initially, Alice has a stones, Bob has b stones, and the third pile contains c stones.

Alice and Bob take turns, with Alice going first. On each turn, the current player may take any number of stones from the third pile, possibly zero, and add them to their own pile.

If both players take zero stones on two consecutive turns, the game ends.

Let A and B be the final numbers of stones Alice and Bob have, respectively. The score of the game is $|A - B|$.

Alice wants to maximize the score, while Bob wants to minimize it. Assuming both players play optimally, find the final score.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

Each test case contains three integers a, b , and c ($0 \leq a, b, c \leq 10^9$) — the initial numbers of stones Alice has, Bob has, and the third pile has, respectively.

Output

For each test case, output one integer — the final score if both players play optimally.

It is important to use a 64-bit integer type, such as `long long` in C++.

input
5 3 6 3 3 6 10 5 5 4 2 5 6 67676767 41414141 998244353
output
3 7 4 3 1024506979

In the first test case, Alice can choose to take 0 stones on her first turn. Bob can then also take 0 stones, so the game ends with piles of sizes 3 and 6. Therefore, the score can be 3. It can be shown that Alice cannot achieve a larger score, and Bob cannot achieve a smaller score.

In the second test case, Alice can take all 10 stones from the third pile on her first turn. The game then ends with piles of sizes 13 and 6, so the score is 7.

C. AND, OR, Sort!

2 seconds, 256 megabytes

You are given a **binary string*** s of length n .

You may perform the following operation any number of times (possibly zero):

- choose an integer i ($1 \leq i \leq n$), and replace s_i with either the **bitwise AND** or the **bitwise OR** of $s_1, s_2, \dots, s_i$.

Note that the bitwise AND or bitwise OR of a single element is equal to the element itself.

Your goal is to make s sorted in non-decreasing order†.

Find the minimum number of operations required to sort s in non-decreasing order.

*A binary string only contains characters 0 and 1.

† If s is in non-decreasing order, then $s_1 \leq s_2 \leq \dots \leq s_n$.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \leq n \leq 2 \cdot 10^5$) — the length of the binary string s .

The second line of each test case contains the binary string s of length n . Each character of s is either 0 or 1.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the minimum number of operations required to sort s in non-decreasing order.

input
6 4 0011 4 1000 5 01000 8 01001101 7 0101010 7 0111101
output
0 3 1 2 3 1

In the first test case, the string is already sorted, so no operations are required.

In the second test case, we can use bitwise OR to change the last three characters to 1, obtaining 1111 in 3 operations.

D. Falling Concrete

2 seconds, 256 megabytes

Vihaan is repairing a road consisting of n sections. The height of the i -th section is a_i .

He has a forklift which can move road sections. In one operation, Vihaan chooses two indices i and j ($1 \leq i < j \leq n$), picks up the j -th section, and moves it backwards to position i .

As the section is carried backwards, one unit of concrete falls from it onto each section it passes over. Then, the carried section is inserted at position i .

More formally, the subarray

$$[a_i, a_{i+1}, \dots, a_{j-1}, a_j]$$

is replaced with

$$[a_j - (j - i), a_i + 1, a_{i+1} + 1, \dots, a_{j-1} + 1].$$

A part of the road is called *flat* if it is a contiguous segment of sections with equal heights.

Vihaan may perform any number of operations, possibly zero.

Find the maximum possible length of a flat part of the road.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of sections of the road.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($n \leq a_i \leq 10^9$) — the initial heights of the sections.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

It can be shown that under the given constraints, the height of every section remains positive after any sequence of operations.

Output

For each test case, print a single integer — the maximum possible length of a flat part of the road.

input
8 5 5 5 9 8 6 6 6 6 6 6 5 5 6 7 8 9 5 9 7 12 10 12 4 4 7 5 8 7 14 9 14 12 8 11 12 5 1000000000 1000000000 1000000000 1000000000 1000000000 8 8 8 12 8 14 10 15 13
output
4 6 1 5 4 2 5 6

In the first test case, Vihaan can move the section at position 4 to position 1:

$$[5, 5, 5, 9, 8] \rightarrow [6, 6, 6, 6, 8].$$

Thus, a flat part of length 4 can be created.

In the fourth test case, Vihaan can perform the following operations:

- 1. Move the section at position 3 to position 1:

$$[9, 7, 12, 10, 12] \rightarrow [10, 10, 8, 10, 12].$$

- 2. Move the section at position 4 to position 3:

$$[10, 10, 8, 10, 12] \rightarrow [10, 10, 9, 9, 12].$$

- 3. Move the section at position 5 to position 3:

$$[10, 10, 9, 9, 12] \rightarrow [10, 10, 10, 10, 10].$$

Thus, the entire road can be made flat.

In the fifth test case, Vihaan can first move the section at position 2 to position 1:

$$[4, 7, 5, 8] \rightarrow [6, 5, 5, 8].$$

Then, he can move the section at position 4 to position 2:

$$[6, 5, 5, 8] \rightarrow [6, 6, 6, 6].$$

Thus, the entire road can be made flat.

E. Prime Destruction

2 seconds, 256 megabytes

You are given a **multiset** a consisting of n positive integers.

You may perform the following operation any number of times (possibly zero):

- Choose an integer $x > 1$ from the multiset and a prime divisor p of x . Remove one occurrence of x from the multiset and add p copies of $\frac{x}{p}$.

You are also given an integer k ($1 \leq k \leq n$). Let $f(k)$ be the minimum number of operations required, starting from the original multiset, until every integer in the multiset is at most k .

Find $f(k)$.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and k ($1 \leq k \leq n \leq 2 \cdot 10^5$) — the initial size of the multiset and the given integer, respectively.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the elements of the multiset.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer $f(k)$.

input
6 1 1 1 6 2 6 6 4 3 2 1 8 1 8 6 4 3 2 1 8 6 12 3 12 10 9 8 7 6 5 4 3 2 1 12 10 9 10 9 8 7 6 5 4 3 2 1 5 5 5 4 3 2 1
output
0 4 25 15 1 0

In the first test case, the only element is already at most k , so no operations are needed.

In the second test case, we can perform the following operations:

- $[6, 6, 4, 3, 2, 1] \rightarrow [2, 2, 2, 6, 4, 3, 2, 1]$,
 $[2, 2, 2, 6, 4, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 4, 3, 2, 1]$,
 $[2, 2, 2, 2, 2, 2, 4, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 1]$,
 $[2, 2, 2, 2, 2, 2, 2, 2, 3, 2, 1] \rightarrow [2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 2, 1]$.

Thus, 4 operations are sufficient. It can be shown that no sequence with strictly fewer operations exists.

F. MEX Replacement

2 seconds, 256 megabytes

You are given a multiset of non-negative integers in compressed form. That is, for each value x_i in the multiset, you know the number of its occurrences y_i .

In one operation, you may choose any non-empty collection of elements currently in the multiset, remove all chosen elements, and insert one element equal to their MEX*.

You may perform the operation any number of times, possibly zero.

Find the maximum integer that can appear in the multiset at any moment.

*The minimum excluded (MEX) of a collection of integers $c_1, c_2, \dots, c_k$ is the smallest non-negative integer x that does not occur in the collection.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of distinct integers in the initial multiset.

Each of the next n lines contains two integers x_i and y_i ($0 \leq x_i \leq 10^9$, $1 \leq y_i \leq 10^9$), indicating that the integer x_i occurs exactly y_i times in the initial multiset.

The integers $x_1, x_2, \dots, x_n$ are pairwise distinct.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output one integer — the maximum integer that can appear in the multiset at any moment.

input
10 4 2 2 0 3 4 1 1 2 1 10 1 1 2 3 3 0 1 2 2 3 1 3 0 4 1 2 2 1 1 0 100 2 2 7 3 1 2 0 1 3 3 3 0 1 1 1 59 1 4 0 5 1 1 2 3 3 1
output
5 10 3 4 4 7 4 3 59 5

In the first test case, the initial multiset is $\{0, 0, 0, 1, 1, 2, 2, 4\}$.

We can replace $\{0, 1, 2\}$ with 3, obtaining $\{0, 0, 1, 2, 3, 4\}$. Then, we can replace $\{0, 1, 2, 3, 4\}$ with 5.

It can be shown that no integer greater than 5 can appear, so the answer is 5.

In the second test case, 10 already appears in the initial multiset, and no greater integer can appear. Thus, the answer is 10.

G. Modular Tree

3 seconds, 256 megabytes

Vihaan has a rooted tree* with n nodes. The tree is rooted at node 1. Each node i has an initial value a_i and a modulus b_i . Let x_i denote the **current** value of node i . Initially, $x_i = a_i$.

Vihaan may perform the following operation any number of times:

- Choose a node u . Let s be the sum of the **current** values of all direct children of u , then replace x_u with $(x_u + s) \bmod b_u$.

After performing any number of operations, Vihaan wants to maximize the sum of the values of all nodes.

Determine the maximum possible sum.

*A tree is an undirected connected graph in which there are no cycles.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of nodes in the tree.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < b_i$) — the initial values of the nodes.

The third line contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$) — the moduli of the nodes.

Each of the next $n - 1$ lines contains two integers u and v ($1 \leq u, v \leq n$) — an edge between nodes u and v .

It is guaranteed that the given edges form a tree.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print one integer — the maximum possible sum of the values of all nodes after performing any number of operations.

input
8 1 3 7 2 0 3 5 4 1 2 3 0 2 3 7 3 4 1 2 2 3 3 0 0 1 5 2 2 1 2 2 3 4 1 2 3 4 10 3 4 5 1 2 1 3 1 4 3 0 1 3 10 2 4 1 2 1 3 5 0 0 1 2 3 12 6 9 3 4 1 2 1 3 2 4 3 5 4 0 999999999 999999999 999999999 1000000000 1000000000 1000000000 1000000000 1 2 1 3 1 4
output
3 7 11 6 18 12 27 3999999996

In the first test case, node 1 has no children, so performing an operation on it does not change its value. Thus, the maximum possible sum is 3.

In the third test case, Vihaan can perform the operation on node 1 three times:

- 1. $[0, 2, 3] \rightarrow [2, 2, 3]$.
- 2. $[2, 2, 3] \rightarrow [4, 2, 3]$.
- 3. $[4, 2, 3] \rightarrow [6, 2, 3]$.

The resulting sum is $6 + 2 + 3 = 11$. It can be shown that no sequence of operations can obtain a larger sum.

In the fourth test case, Vihaan can perform the following operations:

- 1. Perform the operation on node 2:

$[0, 0, 1] \rightarrow [0, 1, 1]$.

- 2. Perform the operation on node 1 four times:

$[0, 1, 1] \rightarrow [1, 1, 1] \rightarrow [2, 1, 1] \rightarrow [3, 1, 1] \rightarrow [4, 1, 1]$.

The resulting sum is $4 + 1 + 1 = 6$. It can be shown that no sequence of operations can obtain a larger sum.

H. Deque Malfunction

3 seconds, 256 megabytes

Vihaan built a **deque** on his own, but it sometimes malfunctions.

Vihaan wants the final contents of the deque, from front to back, to be $[1, 2, \dots, n]$.

You are given an array b of length m . The elements arrive one at a time in the order $b_1, b_2, \dots, b_m$.

Whenever an element b_i arrives, Vihaan must perform exactly one of the following operations:

- push b_i to the front of the deque.
- push b_i to the back of the deque.
- make the deque malfunction and insert b_i at any position of his choice in the deque.

Vihaan may also perform either of the following operations at **any** time:

- remove the element at the front of the deque.
- remove the element at the back of the deque.

After all m elements have arrived and all operations have been performed, the deque must contain exactly $[1, 2, \dots, n]$ in that order.

Determine the minimum number of times the deque must malfunction to create $[1, 2, \dots, n]$, or determine that it is impossible.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($1 \leq n \leq m \leq 2 \cdot 10^5$) — the length of the desired deque and the number of arriving elements.

The second line contains m integers $b_1, b_2, \dots, b_m$ ($1 \leq b_i \leq n$) — the order in which the elements arrive.

It is guaranteed that the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print one integer — the minimum number of times the deque must malfunction.

If it is impossible to obtain the desired deque, print -1 .

input
9
1 3
1 1 1
3 4
2 1 1 2
4 4
1 4 2 3
4 5
4 3 4 2 1
5 8
1 3 2 5 4 3 1 2
6 6
1 6 5 4 3 2
4 6
1 3 2 3 4 3
6 8
2 6 3 5 2 1 4 2
6 8
1 6 2 5 1 5 3 4

output
0
-1
2
0
1
4
0
2
3

In the first test case, Vihaan can perform the following operations:

1. Push the first 1 to the front:

$[] \rightarrow [1]$.

Then, immediately pop it:

$[1] \rightarrow []$.

2. Push the second 1 to the front:

$[] \rightarrow [1]$.

Then, immediately pop it:

$[1] \rightarrow []$.

3. Push the last 1 to the back:

$[] \rightarrow [1]$.

Thus, no malfunctions are required.

In the second test case, the value 3 never appears in b , so it is impossible to obtain the desired deque.

In the third test case, Vihaan can perform the following operations:

1. Push 1 to the back:

$[] \rightarrow [1]$.

2. Push 4 to the back:

$[1] \rightarrow [1, 4]$.

3. Make the deque malfunction and insert 2 between 1 and 4:

$[1, 4] \rightarrow [1, 2, 4]$.

4. Make the deque malfunction and insert 3 between 2 and 4:

$[1, 2, 4] \rightarrow [1, 2, 3, 4]$.

It can be shown that no sequence of operations using fewer than 2 malfunctions can obtain the desired deque. Thus, the minimum is 2. In the fourth test case, Vihaan can perform the following operations:

1. Push the first occurrence of 4 to the front:

$[] \rightarrow [4]$.

Then, immediately pop it:

$[4] \rightarrow []$.

2. Push 3 to the back:

$[] \rightarrow [3]$.

3. Push the second occurrence of 4 to the back:

$[3] \rightarrow [3, 4]$.

4. Push 2 to the front:

$[3, 4] \rightarrow [2, 3, 4]$.

5. Push 1 to the front:

$$[2, 3, 4] \rightarrow [1, 2, 3, 4].$$

Thus, no malfunctions are required.

Hints: Codeforces Round 1122 (Div. 3)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Good Contest (official hint)

Hint: How many participants can possibly have solved all three problems?

B. Three Piles (official hints)

Hint 1: Consider the cases $a \geq b$ and $a < b$ separately.

Hint 2: If Alice ever takes stones, she should take all of them.

C. AND, OR, Sort! (official hints)

Hint 1: If a prefix contains both 0 and 1, what are its bitwise AND and bitwise OR?

Hint 2: A sorted binary string is determined by one split point.

D. Falling Concrete (official hints)

Hint 1: Look at how the height and position of each section change after an operation.

Hint 2: A section moving one position to the right gains 1 height, while a section moving one position to the left loses 1 height.

E. Prime Destruction (official hint)

Hint: Use DP. Let dp_i be the minimum number of operations needed to turn one i into elements which are all at most k .

F. MEX Replacement (official hints)

Hint 1: Work backwards from one copy of the value you want to create.

Hint 2: Process the values from large to small, and let q be the number of copies of the current value that you need.

Hint 3: If you need q copies of x but only have $c < q$, the missing $q - c$ copies of x each force another copy of every smaller value.

G. Modular Tree (official hints)

Hint 1: Process the tree from the leaves upward.

Hint 2: A node can always be brought back to its original value after changing it.

Hint 3: For every node i , its reachable values have the form $(a_i + g_i x) \bmod b_i$ for some integer x . Find g_i .

H. Deque Malfunction (official hints)

Hint 1: We only need to keep one occurrence of every value $1, 2, \dots, n$. What can we do with every other occurrence?

Hint 2: Let t_i be the arrival position of the occurrence of i which stays in the final deque. When can i be inserted without a malfunction?

Hint 3: An element can be inserted normally exactly when t_i is a prefix minimum or a suffix minimum of $t_1, t_2, \dots, t_n$.

Hint 4: Fix the position of the global minimum. The prefix minima and suffix minima can be optimized separately.

Source: <https://codeforces.com/blog/entry/156984>